Arreglos unidimensionales y bidimensionales en C++

Resumen en mis palabras. El código completo está en arreglos.cpp

1. Arreglo unidimensional

Un arreglo es un grupo de casillas del **mismo tipo**, una tras otra en memoria, con un **tamaño fijo** que decido al escribir el código. Las posiciones van de **0 a n-1**.

```
int a[5];                       // 5 casillas con basura
int b[5] = {4, 9, 2, 7, 5};     // con valores
int c[5] = {0};                 // todas en 0
int d[] = {1, 2, 3};            // el tamano (3) se deduce

b[0] = 10;                      // cambiar una casilla
cout << b[4];                   // leer la ultima
```

posición	0	1	2	3	4
valor	4	9	2	7	5

Para recorrerlo uso un `for` con el contador desde 0. Simulación de la suma de {4, 9, 2, 7, 5}:

```
int s = 0;
for (int i = 0; i < 5; i++) s += a[i];
```

i	**a[i]**	**s**
0	4	4
1	9	13
2	2	15
3	7	22
4	5	27

Importante: el arreglo clásico no sabe su tamaño y no avisa si me salgo (`a[7]` en uno de 5 es un error silencioso). Por eso, al pasarlo a una función, mando el tamaño aparte:

```
int suma(const int a[], int n) {     // n = cuantos elementos hay
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}
```

Además un arreglo **siempre se pasa por referencia** (no se copia), así que si la función lo modifica, cambia el original. Por eso uso `const` cuando solo quiero leerlo. Ejemplo que sí modifica, invertir {4, 9, 2, 7, 5}:

```
void invertir(int a[], int n) {
    for (int i = 0; i < n / 2; i++) {
        int temp = a[i];
```

```
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = temp;
    }
}
```

i	intercambia	arreglo queda
0	a[0] con a[4]	5 9 2 7 4
1	a[1] con a[3]	5 7 2 9 4
fin	i = 2 no cumple 2 < 2	5 7 2 9 4

2. Arreglo bidimensional (matriz)

Es un arreglo de arreglos: una tabla con **filas y columnas**. Se declara con dos tamaños, `[filas][columnas]`, y se accede con dos índices.

```
int m[2][3] = {{1, 2, 3},
               {4, 5, 6}};     // 2 filas, 3 columnas

m[0][2];                       // 3  (fila 0, columna 2)
m[1][0];                       // 4  (fila 1, columna 0)
```

	col 0	col 1	col 2
fila 0	1	2	3
fila 1	4	5	6

Para recorrerlo uso **dos ciclos anidados**: el de afuera es la fila (i) y el de adentro la columna (j). El de adentro da todas sus vueltas por cada vuelta del de afuera.

```
for (int i = 0; i < 2; i++) {          // filas
    for (int j = 0; j < 3; j++) {      // columnas
        cout << m[i][j] << " ";
    }
    cout << endl;
}
```

Sumar la fila 1 {4, 5, 6}: con i fijo en 1, j va de 0 a 2: 4, luego 4+5 = 9, luego 9+6 = **15**.

Al pasar una matriz a una función, el **número de columnas es obligatorio** en el parámetro (el de filas no, pero lo mando aparte):

```
int sumaFila(const int m[][3], int fila) {
    int s = 0;
    for (int j = 0; j < 3; j++) s += m[fila][j];
    return s;
}
```

Transpuesta: la posición [i][j] pasa a [j][i]. Una matriz 2x3 se vuelve 3x2, entonces el destino se declara con las dimensiones invertidas:

```
int t[3][2];
for (int i = 0; i < 2; i++)
    for (int j = 0; j < 3; j++)
```

```
        t[j][i] = m[i][j];
```

3. Errores típicos

Error	Solución
Contar desde 1	Los índices van de 0 a n-1
Usar `i <= n`	Usar siempre `i < n`
Intentar leer `a.size()`	No existe en arreglos clásicos: guardar el tamaño en una constante
Confundir `m[i][j]` con `m[j][i]`	Primero la fila, luego la columna
Pasar `int m[][]` a una función	Falta el número de columnas: `int m[][3]`
Declarar sin valores y leer	Las casillas tienen basura: inicializar con `= {0}`

Conclusión

Un arreglo unidimensional es una fila de casillas y uno bidimensional es una tabla de filas y columnas. Con uno recorro con un ciclo, con el otro con dos ciclos anidados. Como el arreglo clásico no conoce su tamaño y no revisa el rango, tengo que llevar yo la cuenta de n y cuidar los índices. Cuando necesito que crezca o que me avise de errores, `std::vector` es la mejor opción.